

ARIA 블록암호 기반 HCTR2 운용 모드의 성능 분석 및 최적화

엄시우, 송민호, 윤세영, 서화정 (한성대학교)

[Current Topic]

연구 배경

1. 연구 배경

✓ AEAD의 필요성

- ▶ AEAD, Authenticated Encryption with Associated Data
- ▶ GCM, CCM 등이 대표적인 AEAD 운용모드
- ▶ 데이터의 기밀성과 무결성을 동시에 보장하는 인증 암호화, TLS 1.3 등 최신 프로토콜의 필수 표준

✓ 기존 GCM의 한계

- ▶ GCM은 암호화할 때마다 고유한 Nonce를 사용하는데, 같은 Nonce를 두 번 이상 사용하면 공격자가 암호문을 분석해 원본 데이터를 복원하거나 인증 태그를 위조할 가능성이 존재
- ▶ Counter 모드 특성상 Nonce가 재사용되면 두 암호문 간의 XOR 연산만으로 평문 정보가 노출될 수 있음

✓ NIST Accordion

- ▶ Nonce 재사용에도 안전하며, 길이 보존 암호화를 지원하는 새로운 운용모드 표준화 프로젝트
- ▶ Accordion 운용 모드에서는 HCTR2를 참조 모델로 활용

1. 연구 배경

✓ AEAD의 필요성

- ▶ AEAD, Authenticated Encryption with Associated Data

본 논문에서는 새로운 Accordion 운용모드에서 사용되는 HCTR2 구조를
한국 표준 블록 암호 ARIA에 적용한 **ARIA-HCTR2**를 구현하고,
이를 기반으로 **Accordion 모드에서의 ARIA 성능 지표**를 제시

✓ NIST Accordion

- ▶ Nonce 재사용에도 안전하며, 길이 보존 암호화를 지원하는 새로운 운용모드 표준화 프로젝트
- ▶ Accordion 운용 모드에서는 HCTR2를 참조 모델로 활용

[Current Topic]

관련 연구

2. 관련 연구 – GCM 모드

✓ GCM – Galois Counter Mode

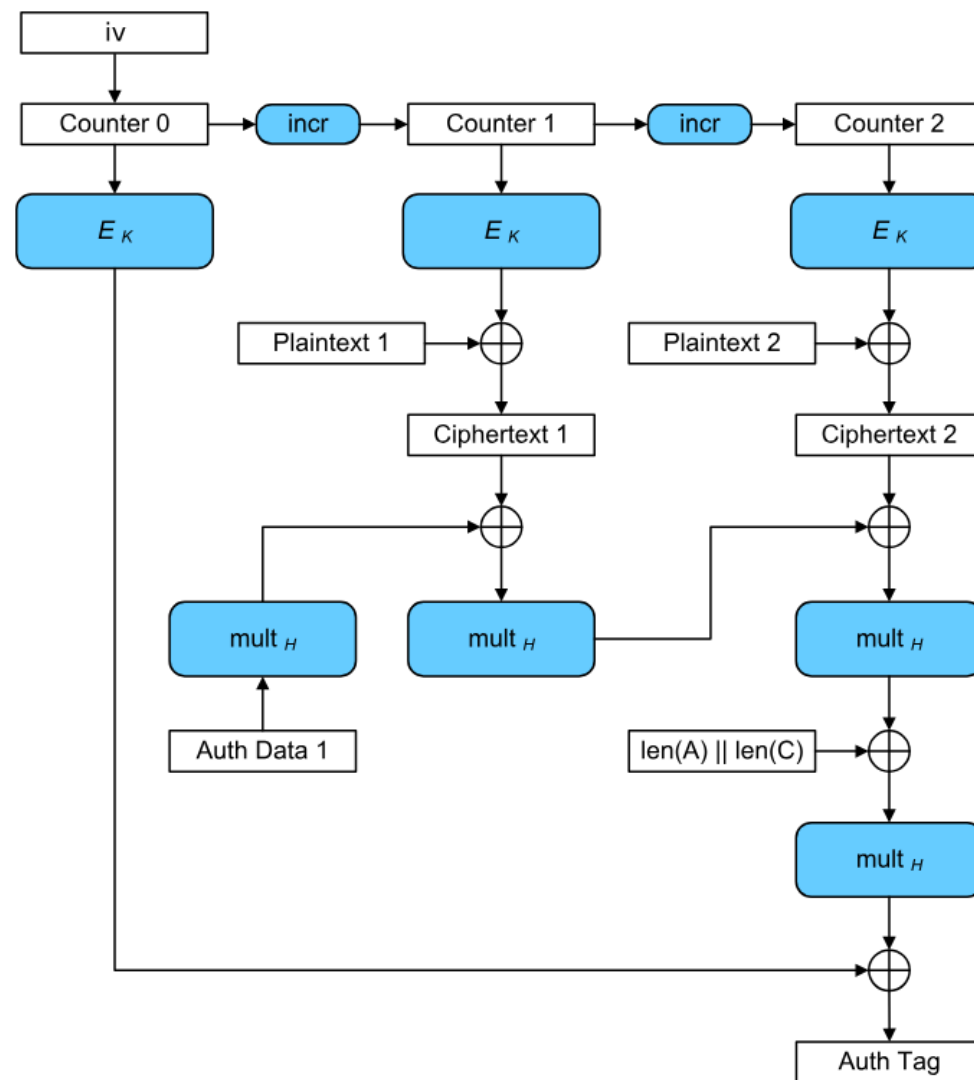
- ▶ CTR 모드 (암호화) + GHASH(인증) 결합 구조
- ▶ 현재 TLS, Isec 등에서 표준으로 사용

✓ 주요 특징

- ▶ CTR 모드 -> 병렬 처리 가능 -> 높은 성능
- ▶ GHASH에 대한 SIMD 명령어 지원
 - ARM NEON PMULL
 - Intel PCLMULQDQ
- ▶ 암호화와 인증을 동시에 수행

✓ 한계점

- ▶ **Nonce 재사용 시 치명적 보안 취약점**
- ▶ 한 번의 Nonce 재사용으로 평문 노출 및 위조 가능



2. 관련 연구 – HCTR2 모드

✓ HCTR2의 특징

- ▶ Google 팀에서 개발한 운용 모드
- ▶ Wide-Block Cipher 구조 (임의 길이 메시지 처리)
- ▶ Hash-Encrypt-Hash 형태의 3단계 구조
- ▶ XCTR 모드 + Polyval 해시 조합

✓ HCTR2의 보안적 장점

- ▶ Nonce 재사용에 대한 안전
- ▶ 강력한 tweakable 암호화 제공
- ▶ 전체 메시지에 대한 Diffusion 보장

✓ HCTR2의 성능적 특징

- ▶ 병렬 처리 가능한 구조
- ▶ Polyval 해시 함수의 효율적 연산 – SIMD 명령어 최적화 가능

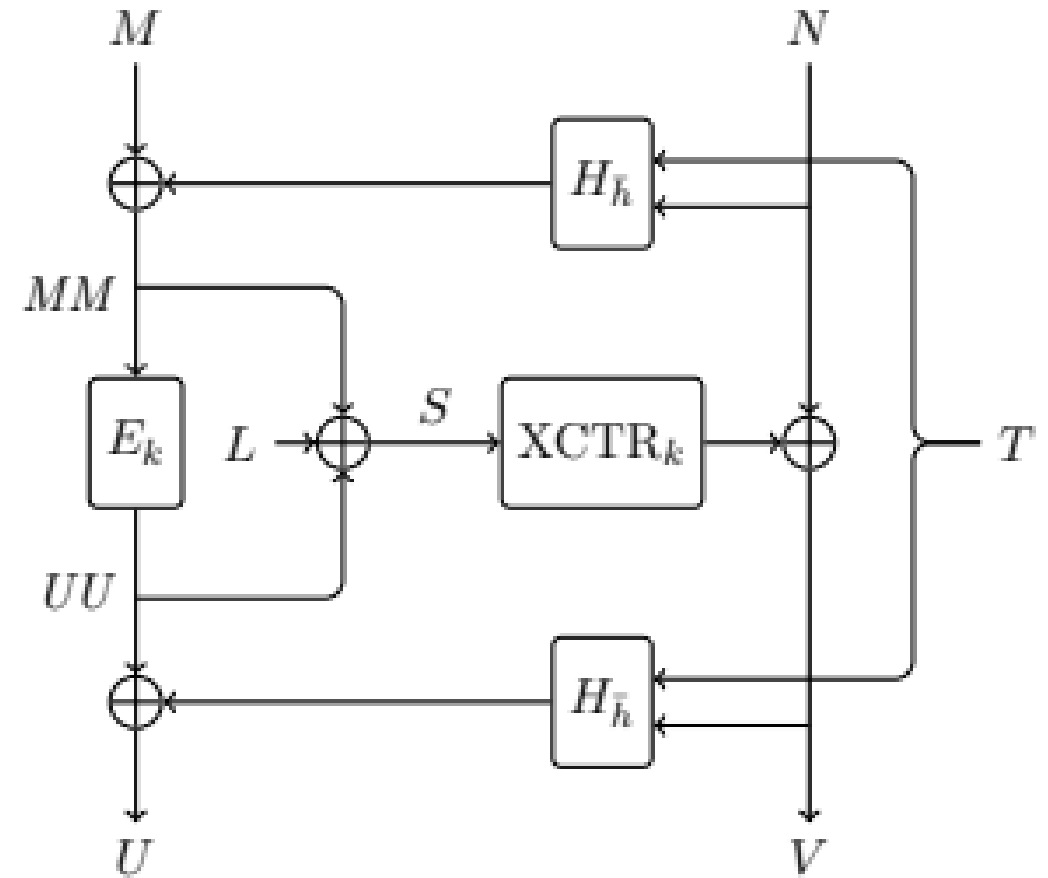
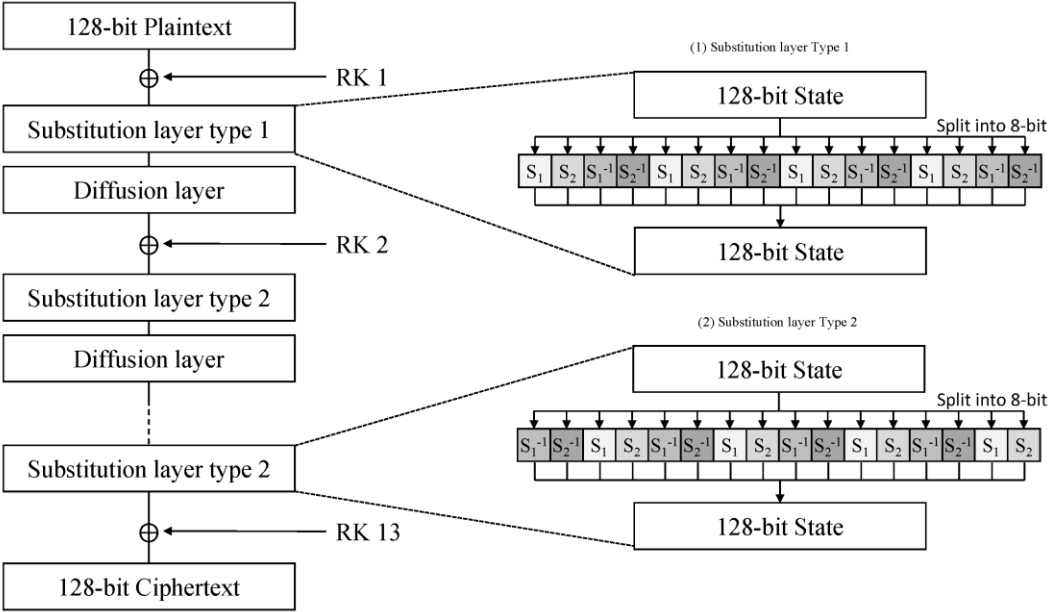


Figure 1: HCTR2

2. 관련 연구 – ARIA 블록 암호

- ✓ Target Block Cipher – **ARIA**
 - ▶ 128비트 크기의 한국 표준 블록 암호
 - ▶ SPN(Substitution Permutation Network) 구조
 - AES와 유사한 블록 암호
- ✓ 최적화 구현 연구
 - ▶ ARMv8 상에서의 병렬 ARIA 최적화 구현[1]
 - 4개 블록의 병렬 연산하는 최적화 구현 논문
 - C 구현 대비 약 4배 빠른 성능을 보여줌



[1] 엄시우, 서화정. (2024). 64-비트 ARMv8 프로세서 상에서의 개선된 병렬 ARIA 최적화 구현. 한국정보통신학회논문지, 28(5), 600-607. 10.6109/jkiice.2024.28.5.600

[Current Topic]

ARIA-HCTR2 최적화 구현

3. ARIA-HCTR2 최적화 구현

- ✓ Github의 공개된 Reference 코드 활용¹⁾
 - ▶ Reference 코드의 경우 AES-HCTR2이 구현되어 있음
 - ▶ AES -> ARIA 로 교체하여 성능 측정에 활용
- ✓ HCTR2의 XCTR 병렬 구현을 통한 최적화
 - ▶ XCTR에 ARM에서의 ARIA 병렬 최적화 구현 적용

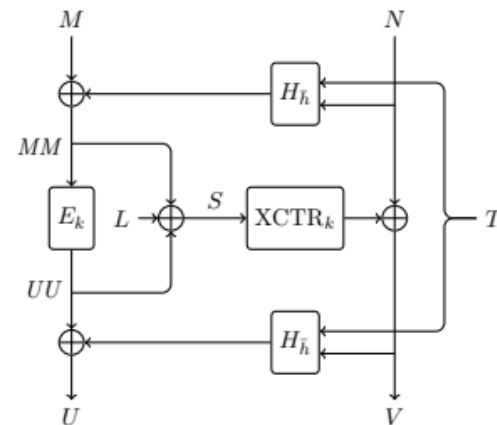
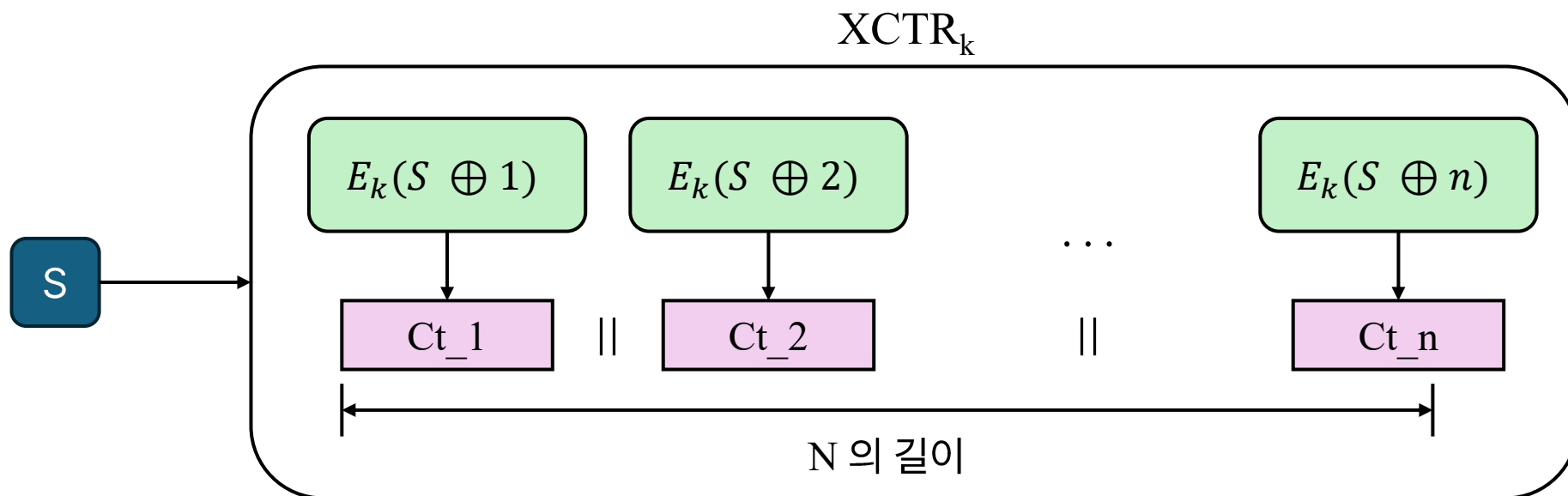


Figure 1: HCTR2



¹⁾: <https://github.com/google/hctr2>

[Current Topic]

성능 평가

4. 성능 평가

Component	Environment A	Environment B
Processor	Apple M1	Apple M4 Pro
OpenSSL Version	3.5.2	3.1.4
Compiler	Clang 17.0.0	Clang 17.0.0
Comparison Targets	ECB, CBC, CTR, GCM, HCTR2 (Hardware Accelerated)	

4. 성능 평가

- ✓ OpenSSL의 Speed를 활용하여 ECB, CBC, CTR, GCM의 성능을 측정
 - 하드웨어 가속기와 같은 최적화가 적용되지 않은 일반 성능과 최적화 성능을 모두 비교
 - OPENSSL_armcpa="-0x1"

운용모드	M1	M4 Pro
ECB	205.85	306.77
CBC	178.94	256.74
CTR	180.91	268.20
GCM	119.44	184.67
HCTR2	50.44	65.58



일반 성능 (MB/s)

운용모드	M1	M4 Pro
ECB*	204.48	306.63
CBC*	177.76	258.81
CTR*	180.75	270.81
GCM*	198.16	309.98
HCTR2*	233.95	396.75



최적화 성능(MB/s)

4. 성능 평가

✓ 일반 성능 vs SIMD 최적화 성능 비교 (M4 Pro)

- ▶ 약 6배 정도의 성능 향상



✓ GCM vs HCTR2 최적화 성능 비교 (M4 Pro)

- ▶ 약 28% 빠른 성능



5. 결론

- ✓ ARIA-128에 대한 새로운 HCTR2 모드 적용
 - ARM NEON SIMD를 활용한 XCTR 병렬 구현 적용
 - 최신 Apple Silicon(M1/M4)에서 다른 운용 모드와의 성능 분석
 - OpenSSL의 Speed를 통해 측정된 성능에서 보다 약 28% 좋은 성능 결과를 확인

- ✓ 향후 연구
 - Accordion의 Acc128 뿐만 아니라 Acc256, BBBAcc에서의 성능 분석
 - GPU 기반의 대량 병렬 연산을 통한 성능 분석

감 사 합 니 다